

## Part 4(d): Why the three methods agree

**Same problem.** Backward DP, value iteration (VI) and policy iteration (PI) are applied to one and the same model: identical state grids, dynamics  $e_{k+1} = e_k + \eta_k - \Delta r_k$ ,  $\eta_{k+1} = \eta_k + u_k + w_k$ , stage cost  $g$ , terminal cost  $g_N$ , disturbance distribution  $p(w)$  and admissible sets  $\mathcal{U}(\eta)$ . The augmented state  $z = (k, e, \eta)$  with the zero-cost absorbing state  $\Delta$  used by VI and PI is only a rewriting of the finite-horizon problem as a stationary total-cost problem: from  $(k, e, \eta)$  with  $k < N$  the cost  $g$  is charged and the state moves to  $(k+1, \cdot, \cdot)$ ; from  $(N, e, \eta)$  the cost  $g_N$  is charged once and the state moves to  $\Delta$ , where  $J(\Delta) = 0$ .

**Unique fixed point.** All three methods compute the solution of the same Bellman equation

$$J^*(z) = (TJ^*)(z), \quad (TJ)(k, e, \eta) = \min_{u \in \mathcal{U}(\eta)} \left[ g(e, \eta, u) + \sum_w p(w) J(k+1, e + \eta - \Delta r_k, \eta + u + w) \right], \quad (1)$$

with  $(TJ)(N, e, \eta) = g_N(e, \eta) + J(\Delta)$  and  $(TJ)(\Delta) = 0$ . Because every transition increases  $k$  by one until  $\Delta$  is reached, the equation determines  $J^*$  uniquely layer by layer from  $J^*(N, \cdot) = g_N$  backwards. Hence there is exactly one optimal value function, and the methods differ only in how they reach it:

- **Backward DP** solves the equation directly, one layer at a time ( $k = N-1, \dots, 0$ ).
- **VI** applies  $T$  repeatedly from  $J \equiv 0$ . After  $s$  sweeps the layers  $k \geq N+1-s$  are exact, so  $J = J^*$  after 13 sweeps and the residual is 0 in sweep 14.
- **PI** produces policies with  $J_{\mu_{i+1}} \leq J_{\mu_i}$  (policy improvement theorem). Since there are finitely many policies, it stops at a policy with  $TJ_\mu = J_\mu$ , i.e.  $J_\mu = J^*$ .

**Exact arithmetic on the same grid.** The grids  $\mathcal{X}_k$  contain every successor (Part 1), and all successors are integer grid points, so no clipping or interpolation error is introduced by any method. Each method evaluates the same expressions on the same grid values, which is why the observed maximum difference from backward DP is exactly 0 and not merely small.

**Same tie-breaking rule.** Equal values do not by themselves imply equal policies: when two controls give the same cost, either is optimal. All three methods select controls with the same function (**greedy**), which takes the first control in the order  $-1, 0, 1$  whose cost is within  $10^{-12}$  of the minimum. Therefore the extracted policies are identical, with 0 disagreements over the 2760 decision states.

## Bellman backup versus policy evaluation

Both operations are built from the same one-step quantity

$$Q(J, z, u) = g(e, \eta, u) + \sum_{w \in \{-1, 0, 1\}} p(w) J(k+1, e + \eta - \Delta r_k, \eta + u + w), \quad (2)$$

implemented as

```
def Q(J, z, u):
    k, e, eta = z
    e_next = e + eta - dr[k]
    return g(e, eta, u) + sum(p * J[(k + 1, e_next, eta + u + w)] for w, p in W)
```

**Bellman backup** (bellman\_T in VI; greedy in backward DP, VI policy extraction and the PI improvement step):

```
TJ[z] = min(Q(J, z, u) for u in admissible(eta))      # bellman_T

qs = {u: Q(J, z, u) for u in admissible(z[2])}        # greedy
best = min(qs.values())
```

It computes  $(TJ)(z) = \min_{u \in \mathcal{U}(\eta)} Q(J, z, u)$ : every admissible control is tried and the cheapest is kept. It answers *what is the best achievable cost from z*.

**Policy evaluation** (evaluate\_policy in PI):

```
J[z] = Q(J, z, mu[z])                                # evaluate_policy
```

It computes  $(T_\mu J)(z) = Q(J, z, \mu(z))$  with the control fixed by the policy, in one backward pass from  $J_\mu(N, \cdot) = g_N$ . It answers *what does policy  $\mu$  cost from z*.

|                           | Bellman backup                                 | Policy evaluation              |
|---------------------------|------------------------------------------------|--------------------------------|
| Operator                  | $T: \min_{u \in \mathcal{U}(\eta)} Q(J, z, u)$ | $T_\mu: Q(J, z, \mu(z))$       |
| Controls per state        | all admissible (1–3)                           | one, $\mu(z)$                  |
| Minimization              | yes                                            | no                             |
| Fixed point               | optimal values $J^*$                           | policy values $J_\mu \geq J^*$ |
| $Q$ -evaluations per pass | 4968                                           | 2760                           |
| Used in                   | DP, VI, PI improvement                         | PI evaluation                  |

Table 1: Bellman backup versus policy evaluation.

In short, the only difference is the minimization: the Bellman backup optimizes over the control, whereas policy evaluation applies the control prescribed by a given policy. PI alternates the two: it evaluates the current policy with  $T_\mu$  and then improves it with the greedy Bellman backup.